

Discourse2Draft: An AI powered structured document writing tool

Amlan Talukder^{1,3*}, Parker K. Combs^{2,4}, Charles P. Schmitt³, David M. Reif²
and Scott S. Auerbach²

¹ Office of Data Science (ODS), NIH/NIEHS, Durham, NC, United States

² Predictive Toxicology Branch (PTB), Department of Translational Toxicology (DTT), NIH/NIEHS, Durham, NC, United States

³ 22nd Century Technologies, McLean, VA, United States

⁴ Axle Informatics, Rockville, MD, United States

Correspondence*:

Amlan Talukder
amlan.talukder@nih.gov

2 ABSTRACT

We introduce *Discourse2Draft*, a model-agnostic, retrieval-augmented generation (RAG) based application that converts a user-provided query into an exhaustive structured outline and, from that outline, into a polished long-form draft, using context from either user provided documents or literature downloaded from *PubMed* or both. The system can also take the structured outline directly from the user. The system extracts the hierarchical structure from the outline (either AI-generated or user provided) and detects positions to write contents. It then orchestrates prompts to large-language models (LLMs) to extract core ideas from the corresponding section headers and previously written text for every writable position, fill that position with cohesive prose with context taken from user provided documents or online literature, and automatically weave source-anchored citations throughout the text. The draft with associated references and the bibliography can be exported in docx, LaTeX and Markdown formats, ensuring full transparency and downstream editability. By coupling multi-vendor LLM flexibility with traceable knowledge retrieval and granular certainty metrics, *Discourse2Draft* streamlines the journey from informal textual query to publication-quality writing while keeping scientific rigor and author oversight at the forefront.

Keywords: Document Writing, Deep Research, RAG, LLM, AI, AI Workflow

1 INTRODUCTION

The rapid proliferation of large-language models (LLMs) has catalyzed a new generation of “deep research” platforms that promise to accelerate literature discovery, hypothesis generation, and scientific writing. Tools such as Semantic Scholar’s TLDR feature ((Cachola et al., 2020)), Elicit’s question-answering workflow ((Elicit, 2023)), Research Rabbit’s citation-network visualizations ((ResearchRabbit, 2025)), and Scholarcy’s automated paper digests ((Scholarcy, 2026)) exemplify the growing ecosystem of systems that summarize findings, surface related work, or extract structured information from vast corpora. More recently, retrieval-augmented generation (RAG) frameworks—including GitHub Copilot for code and Scite’s Smart Citation Generator ((Nicholson et al., 2021))—have blended local knowledge bases with

generative text to create context-aware outputs that more closely mimic domain expertise. Despite these advances, most existing solutions fail to provide a structured outline as per user intention and authentic references to the content. Many of these tools focus on gathering basic literature information (title and abstract) on a topic, often powered by a simple internet search and/or limited number of document summarization or narrow task support; few enable researchers to generate full, publication-grade manuscripts with structured user provided outline while preserving traceability to source material and providing explicit measures of factual certainty.

Discourse2Draft addresses this gap by offering an open-source, end-to-end, query-to-manuscript pipeline that is simultaneously transparent, customizable, and model-agnostic. It provides full control on the outline of a manuscript (or any structured document) and proper content citations of authentic publications downloaded from a rich and trusted literature database, *PubMed*. After ingesting a user-supplied query, the application uses LLM to shape an outline for the manuscript. It then writes each section of the outline using relevant information from user-provided documents or literature from *PubMed* or both as LLM context using RAG approach. *Discourse2Draft* also exposes granular controls like domain, writing style, size of the written document which should allow authors to modulate both the breadth and depth of the output. By coupling controls on both structure and content of a document with authentic references, *Discourse2Draft* empowers scientists to tailor AI-generated manuscripts to diverse journal formats, grant applications, or internal reports without sacrificing rigor or authorial oversight. *Discourse2Draft* also provides manageability by allowing users to regenerate sections as needed and download the output and references in the popular varieties of editable document formats.

2 METHODS

2.1 User account and settings

Discourse2Draft can be accessed without creating an account. But a user account is helpful to store the generated and uploaded documents and associated settings. The settings include API key and URL of the LLM provider, temperature of the LLM and generic instructions for the content generation.

2.2 User inputs

2.2.1 Query and outline

Discourse2Draft offers flexibility in terms of user interactions. As an input, it takes either a user query (e.g. Write a review article on Quantum Computing.) or a structured outline of the draft following a set of rules mentioned in the next paragraph. With structured outline, user has more control on the generated content as it strictly follows the user provided outline. If a user query is provided as input, it first creates a structured outline and then starts writing the content. So, creating a structured outline is the first essential step in *Discourse2Draft*. The user can make changes to the generated outline with an interactive user interface (UI).

Query tab: User can provide a query and upload with an reference file (if necessary). Next step is to click the “Create outline” button, which will generate a structured outline on the query considering the reference file (if provided).

Structured Outline tab: User can directly provide a structured outline following *Discourse2Draft* provided guidelines. This helps the AI model to parse the structured outline and identify places where it

needs to generate contents. It also helps the AI model to extract the pre-written summary of the content which will be useful to maintain the continuity in generating the new content.

Structured Outline Guidelines: The structured outline must maintain the standard Markdown section hierarchy with the number of “#”s, where the each hierarchy level header is preceded by “#”s. One “#” means the topmost level (which typically is the title of the outline) and consistent increment in the number of “#”s would ensure the sub-levels, e.g. two “#”s mean the second level hierarchy (e.g. Abstract, Introduction etc.) and so on. An outline must begin with one “#” (top-level hierarchy). The position where the AI needs to write the content has to be specified with a “<content>” tag. User can add additional instructions for content generation under each section (or sub-section etc.) enclosing the instructions within “<Instructions>” and “</Instructions>” tags (Fig. 1).

2.2.2 Document upload

Discourse2Draft has the facility to use user-uploaded documents during content generation. The documents can be uploaded as “text (TXT)”, “Microsoft Word (DOCX)” or “PDF” formats from the user machine on the “Upload documents” panel in the bottom left corner of the UI. The uploaded documents can later be attached to a new file for content generation. The uploaded documents ids are stored in a relational database associated with the user account (if the user signed in) or the active session (if the user did not sign in).

2.3 Content generation

2.3.1 Outline processing

To write a document, *Discourse2Draft* either takes a structured outline from the user or creates one based on the user provided query. It then breaks the outline into a hierarchical structure (Fig. 1), validates it and detects the first position in the outline where it needs to write (denoted by the “<content>” tag). It then gathers the header chain specified by “#”s of that position from the topmost level (title → section → subsection → sub-subsection etc.) and the generated contents so far to ensure the relevance and consistency in content generation. It also gathers the instructions for the corresponding section, if it is provided within “<Instructions>” and “</Instructions>” tags. It then detects if the outline has an “Abstract” section or a section with a similar header e.g. “Summary”, “Synopsis”, “Outline” etc. *Discourse2Draft* does this detection with an AI call. If the writable position is part of an “Abstract” section, it saves it to write at the end and move on to the next section. To write in a position, *Discourse2Draft* passes the corresponding section header chain and the summary of the previous contents to a suitable AI workflow (described later in this section). AI model generates the content and a document content summary up to the current position, which can be used to generate the next content (Fig. 2).

2.3.2 AI workflows for content generation

Discourse2Draft follows different AI workflow based on user needs (Fig. 2). Each workflow uses three inputs; the section header chain of the current position, the summary of the previously written contents and user-provided instructions for this section extracted from the “<Instructions>” and “</Instructions>” tags. Each workflow generates two outputs; the AI-generated content and the summary of the overall content (joins the generated content with the previous content summary). In the beginning of an AI workflow, it summarizes the previously written contents, if it contains over 500 tokens to ensure that the number of context-tokens does not exceed the limit of underlying LLM. This step is usually redundant as the input already considers the summarized version of the previously written content. But if the AI fails to

summarize the content in the last step, this step helps to double check and keep the number of tokens within the threshold.

Basic: This workflow generates content from the training data of associated LLM, based on the headers and previously written content summary. This workflow is used as default.

RAG: This workflow considers context from user provided documents. When a user provided document is attached to this workflow, the document is first divided into chunks of 1,000 characters of text with an overlap of 200 characters and then each chunk of text is converted into an embedding vector (called “vector document”) and saved into a vector database table (called “collection”), here we refer it as *document collection*. To generate content for a file with attached user documents, *Discourse2Draft* follows the “RAG” workflow, where the underlying LLM analyzes the previous content summary and current section headers to generate 10 semantically independent key-phrases relevant to the topic. Then each key-phrase is searched in the vector database collection to find 5 semantically similar “vector documents”. The key-phrases and the retrieved texts from the “vector documents” are combined and passed to the LLM as context. Finally, LLM considers the provided context along with its training data to generate the final content.

Literature: Apart from the user uploaded documents, *Discourse2Draft* also has the ability to gather content from online literature database. This ability is turned on by setting “Literature Search” on at the top of the content panel (Fig. 3). After that, *Discourse2Draft* continues the similar steps in the “RAG” workflow, but replaces the vector database search with literature search. Similar to “RAG” workflow, it starts with analyzing the previous content summary and the current section header to generate 5 key-phrases, uses each key-phrase to query against the online literature database, picks 2 best match articles and extracts the first 20000 number of letters from each article. In this version of *Discourse2Draft*, we only use the open access *PubMed Central* database as the literature database ((Roberts, 2001)). The extracted contents of the literature is then processed in the same way as the “RAG” workflow and saved in the vector database collection, which we refer here as (*literature collection*). The rest of the steps are the same as the “RAG” workflow.

RAG + Literature: This workflow, as the name suggests, merges both “RAG” and “Literature” workflows. Here each key-phrase, generated after analyzing the previous contents and the current section headers, will be used for both searching the *document collection* and the *literature collection* of the vector database to retrieve semantically similar text chunks from both sources. The text chunks are then passed to the LLM as context. In this way, the LLM considers the provided context from both uploaded documents and relevant literature search along with its training data to generate the final content.

2.3.3 Other AI workflows

Other than content generation, *Discourse2Draft* uses AI workflows in outline generation and formatting and abstract detection and writing. These workflows are relatively simpler than the content generation workflows and are driven mainly with prompt engineering. The outline generation workflow works when the user provides a text based query, as input, with or without the reference file. If a reference file is provided with the query, the outline generation workflow iteratively summarizes the content from the reference file and extracts the gist that fits within LLM context token limit. When the user provides the structured outline directly as input, the outline formatter workflow is used to fix the outline, if needed. The abstract detection workflow is applied to find if the outline has an “Abstract” or a section with similar header e.g. “Summary”, “Synopsis”, “Outline” etc. If there is an abstract, *Discourse2Draft* writes it using the abstract writer workflow after the entire document is written.

148 2.3.4 Referencing

149 When either of “RAG”, “Literature” and “RAG + Literature” workflows is used, *Discourse2Draft* cites
150 the generated text with references from the user provided documents or online literature. It maintains
151 correctness and order of every reference that is cited in the generated document. When an uploaded
152 document is attached or a literature is searched, a record with the metadata on the document or literature is
153 created in a relational database. In the above three workflows, LLMs use text chunks retrieved from the
154 vector database as context to generate the final content. Each text chunk is tagged with the corresponding
155 record identifier (id) from the relational database. Each text chunk associated with the id is provided to
156 the LLM as context. The LLM is instructed to cite the id of a text chunk at the end of every group of
157 sentences it generates using that text chunk. The cited id is then programmatically processed to create
158 citation at the specified position of the content. A list of references of the cited documents and literature is
159 shown in American Psychological Association format (APA) format at the end of the document under the
160 “References” section (Fig. 3).

161 2.4 Modification of content

162 *Discourse2Draft* provides features that allow user to modify each paragraph of the content after content
163 generation. User can either expand or rephrase a paragraph with specific instructions or completely remove
164 it from the content. The controls for regeneration are shown on the content bar when a paragraph is selected
165 for modification (Fig. 3).

166 2.5 Concept Map

167 Each document is associated with a concept map which is a tree of key-phrases explored by AI to generate
168 content under a section. This map should help user get an overview of the document without reading it. It
169 should also help user validate the thoughts of the AI model during content generation (Fig. 4). The concept
170 map can be accessed via the “Concept Map” button at the top of the content panel.

171 2.6 Custom instructions for AI

172 *Discourse2Draft* follows a set of regulation with the highest priority that are built-in as system prompt.
173 User can add instructions on top of that in the settings menu that apply on the files generated under the
174 user profile. As mentioned earlier, user can also add instructions inside the structured outline within
175 “<Instructions>” and “</Instructions>” tags. These instructions are section specific and are appended
176 with the instructions provided in the user profile settings. Other than these instructions, user can also add
177 instructions to regenerate a paragraph of a file content. These instructions are appended at the end of the
178 other user instructions.

179 2.7 Outputs

180 *Discourse2Draft* provides the facility to download the generated document in Markdown (.md), Microsoft
181 Word (.docx) and LaTeX (.tex) formats. Markdown, Microsoft Word formatted files contain the references
182 inside the file. For the LaTeX formatted file, the bibliography of the associated references can be downloaded
183 separately as “bibtex” format. The download options are shown on the right side of the outline panel, in the
184 generated documents panel and the expanded view of generated documents panel, which can be opened by
185 clicking the expand button on the top right corner of the panel. The concept map of a document can be
186 downloaded in JSON format.

2.8 Technology Stack

Discourse2Draft was built using python programming language. The front-end of *Discourse2Draft* was developed using python-shiny package and JavaScript. For the relational database in the back-end PostgreSQL was used. LangChain and LangGraph packages were used to create and manage the AI workflows. Chroma was used as the vector database (Fig. 5).

2.9 Installation

Discourse2Draft code can be downloaded from GitHub (<https://github.com/amlantalukder/Discourse2Draft>). The repository contains step by step instructions to install it.

3 EVALUATION

We evaluated *Discourse2Draft* by comparing it with two state-of-the-art deepresearch AI tools: *OpenAI deep research* ((OpenAI, 2025)) and *Manus AI* ((Shen et al., 2025)) (Lite version) adopting LLM-as-a-judge approach. As documents to generate, we picked three topics from biomedical domain space; “CRISPR-based therapeutic genome editing for inherited blood disorders”, “Phthalates Toxicity” and “Health Impacts of Per- and Polyfluoroalkyl Substances (PFASs): A Comprehensive Review”. Each of these topics are well researched in the respective domain, thus has many peer-reviewed articles in *PubMed Central*. We generated contents on the three topics with the same queries for the three tools. Each query contains an outline which section hierarchy so that the tools can maintain the structure and hence can be compared with *Discourse2Draft*. For *Discourse2Draft*, we used the “literature search” workflow with GPT-5, o1 and o1-mini from OpenAI as the underlying LLMs to generate the contents on the three topics in the respective order. As evaluator, we used two AI agents using o3 from OpenAI and Gemini 3.1 Pro from Google to score the contents of each section of the three documents within 0 to 100 based on the following criteria:

Relevance: Is the content relevant to the provided section header and the previously written content summary (if provided)?

Continuity: Is the content writing style continuous with the previously written content summary? The highest score (100) is given if previously written content summary is not provided. This criterion is not used to score “References”.

Non-repetitiveness: Is the content devoid of repetitiveness compared with the previously written content summary? The highest score (100) if previously written content summary is not provided.

Specificity: Is the content devoid of hallucination?

Across the evaluated documents, *Discourse2Draft* generally maintained close alignment with section headers and thematic expectations highlighted by the *relevance* scores. This indicates that *Discourse2Draft* effectively interpreted the preset instructions and produced content that adheres to the intended topical scope, a critical requirement for scientific and review-style writing. *Discourse2Draft* also demonstrated solid performance in maintaining *continuity*, despite using a short summary on the previous content controlled with maximum tokens threshold (500). Its outputs tended to preserve a coherent narrative flow and stylistic consistency across sections, suggesting that it effectively leverages contextual inputs. This is especially important in multi-section biomedical documents, where logical progression and conceptual linkage between sections enhance readability and scholarly quality. Although not as strong as the previous two criteria, in terms of *non-repetitiveness* *Discourse2Draft* appeared capable of avoiding redundant phrasing and content overlap with previously generated sections. This contributes to more concise and

information-dense writing, which is desirable in technical domains where redundancy can obscure key insights.

To highlight some of the weaknesses, *Discourse2Draft* exhibited limitations in specificity, particularly in complex biomedical contexts. Instances of overgeneralization and occasional hallucination-like behavior suggest that the model may struggle with grounding its outputs in precise, verifiable scientific detail. This is a critical shortcoming, as specificity and factual accuracy are paramount in biomedical writing. Another observed weakness is inconsistent performance across document sections and topics. While *Discourse2Draft* performs well in structurally straightforward sections (e.g., introductions or summaries), its performance may degrade in more technically demanding sections such as mechanistic explanations or regulatory analyses. This variability suggests that the model's internal knowledge integration and reasoning depth may not be uniformly robust across domains. Although these shortcomings can be overcome by providing more information to the AI workflow. This can be done by revising the user provided instructions and/or increasing the number of tokens in the previous content summary, the number of manuscripts and the content size of each manuscript, which are 500, 2 and 20000 respectively in the default settings. Also, *Discourse2Draft* used a simplistic workflow to download literature documents from only one source (*Pubmed Central*), while the other two proprietary tools use networks of agents ((OpenAI, 2025; Shen et al., 2025)) to do the deep research from a wide variety of sources from internet. Despite these limitations, *Discourse2Draft* was able to maintain a competitive score on all the above-mentioned criteria, and consistently higher scores on the "References".

4 DISCUSSION

We aimed to develop an open-source, AI-powered document generation tool that can efficiently and reliably help researcher generating drafts balancing flexibility with complete content control. *Discourse2Draft* can generate structured contents like other numerous tools available now or appearing everyday. But unlike most of the AI tools, it allows user to maintain control on the structure of the content. The references inserted in the content are devoid of hallucinations as they are cross-referenced and validated during content generation. *Discourse2Draft* also demonstrates a balanced profile in structural alignment, coherence, and non-redundancy, making it well-suited for generating organized draft documents. However, its limitations in specificity and depth of scientific reasoning constrain its effectiveness for high-stakes biomedical writing tasks. As a future improvement, we will focus on enhancing factual grounding, reducing hallucinations, and strengthening domain-specific reasoning capabilities to better compete with leading deep research AI systems. In the current settings, the underlying workflows of *Discourse2Draft* were kept simple to maintain a balance between speed and accuracy. We used *PubMed Central* free full text literature database which primarily focuses on the articles published in biomedical domain. For other domains, we plan to add more online databases that provide open access to the full text of an article (e.g. Web of Science, Scopus etc.). *Discourse2Draft* can serve the purpose of generating a variety of structured documents in academia e.g. manuscripts for research article, literature review, research proposals, thesis dissertation, case study, systematic review etc. The code-base is provided as open source to the community to be extended and perfected as needed.

CONFLICT OF INTEREST STATEMENT

The authors declare that this work was done in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

AUTHOR CONTRIBUTIONS

SA and CS conceptualized the idea. PC designed and managed the AI infrastructure and database support. AT designed and implemented the tool. AT and SA contributed in manuscript writing. PC, CS and DR contributed with feedbacks and revisions.

FUNDING

This research was supported by the NIH, National Institute of Environmental Health Sciences, through Intramural Research Project ZIAES103385.

SUPPLEMENTAL DATA

The documents generated by *OpenAI deep research*, *Manus AI* and *Discourse2Draft* are provided as supplementary data.

REFERENCES

- Cachola, I., Lo, K., Cohan, A., and Weld, D. S. (2020). Tldr: Extreme summarization of scientific documents. *ArXiv abs/2004.15011*
- [Dataset] Elicit (2023). Elicit: The ai research assistant
- Nicholson, J. M., Mordaunt, M., Lopez, P., Uppala, A., Rosati, D., Rodrigues, N. P., et al. (2021). scite: A smart citation index that displays the context of citations and classifies their intent using deep learning. *Quantitative science studies* 2, 882–898
- [Dataset] OpenAI (2025). Introducing deep research
- [Dataset] ResearchRabbit (2025). Researchrabbit (version 2025-11-18) [search tool]
- [Dataset] Roberts, R. J. (2001). Pubmed central: The genbank of the published literature
- [Dataset] Scholarcy (2026). Scholarcy - ai summarizer
- Shen, M., Li, Y., Chen, L., and Yang, Q. (2025). From mind to machine: The rise of manus ai as a fully autonomous digital agent. *arXiv preprint arXiv:2505.02024*

FIGURE CAPTIONS

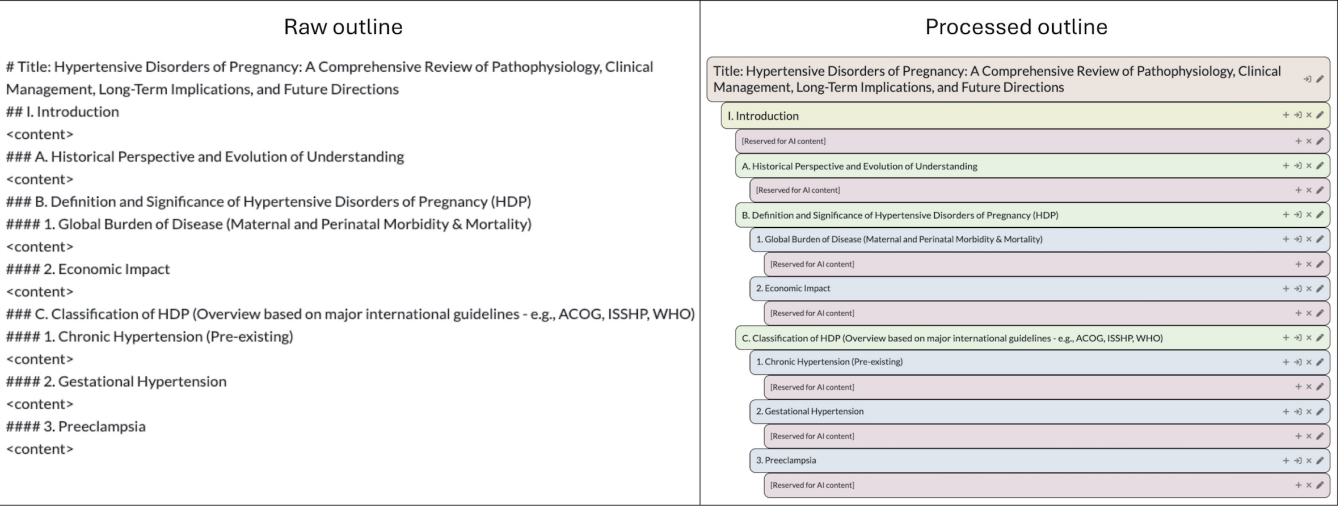


Figure 1. Raw outline input format of content creation. The left panel shows an example of the structured outline with the necessary syntax. The right panel shows the hierarchy of how the outline is processed from the raw text shown in the left panel.

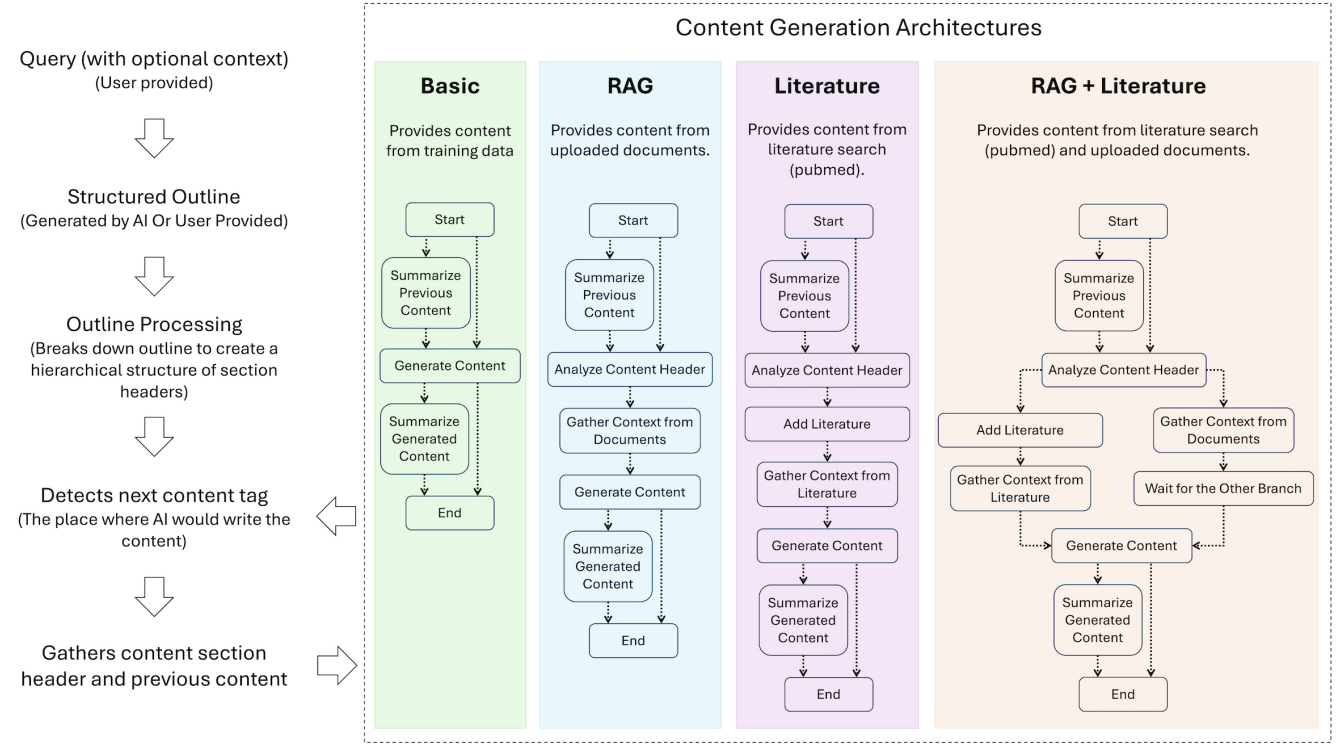


Figure 2. Behind-the-scene user input processing steps and different AI workflows.

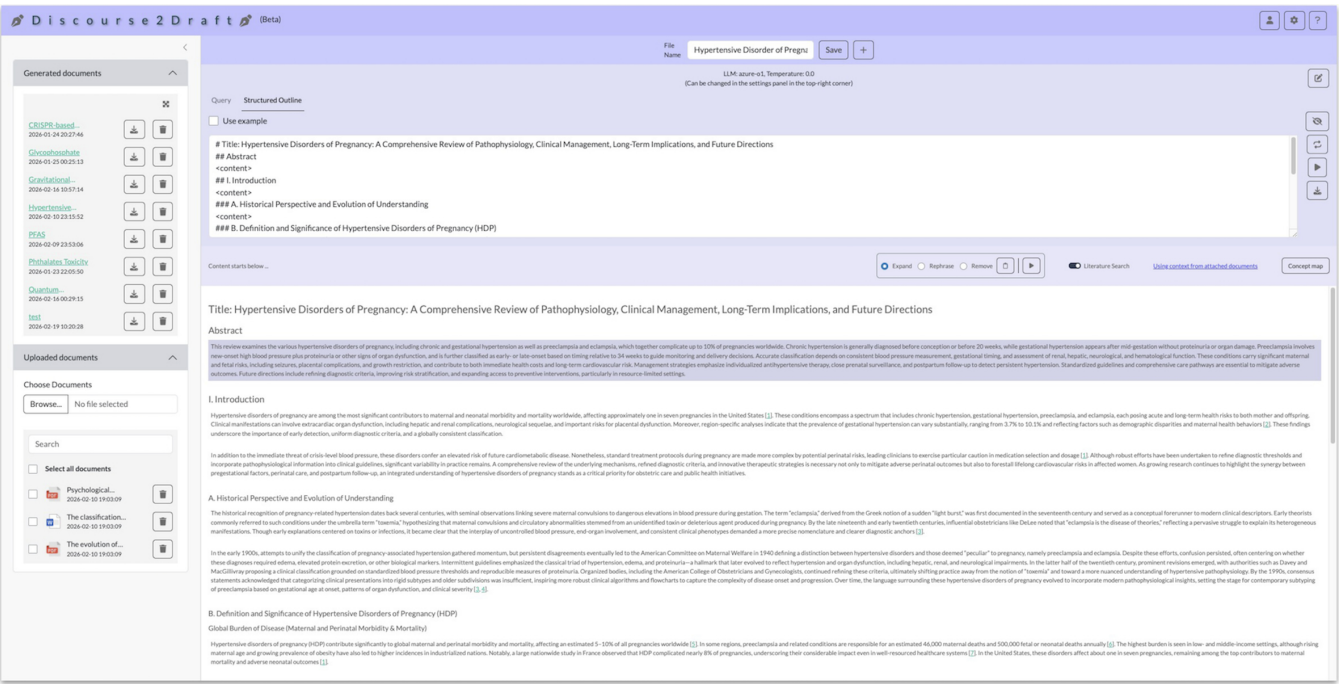


Figure 3. Content generation window of the app. The top left and bottom left panels show the generated and user uploaded files. The top right panel contains the outline. The bottom right panel contains the generated content.

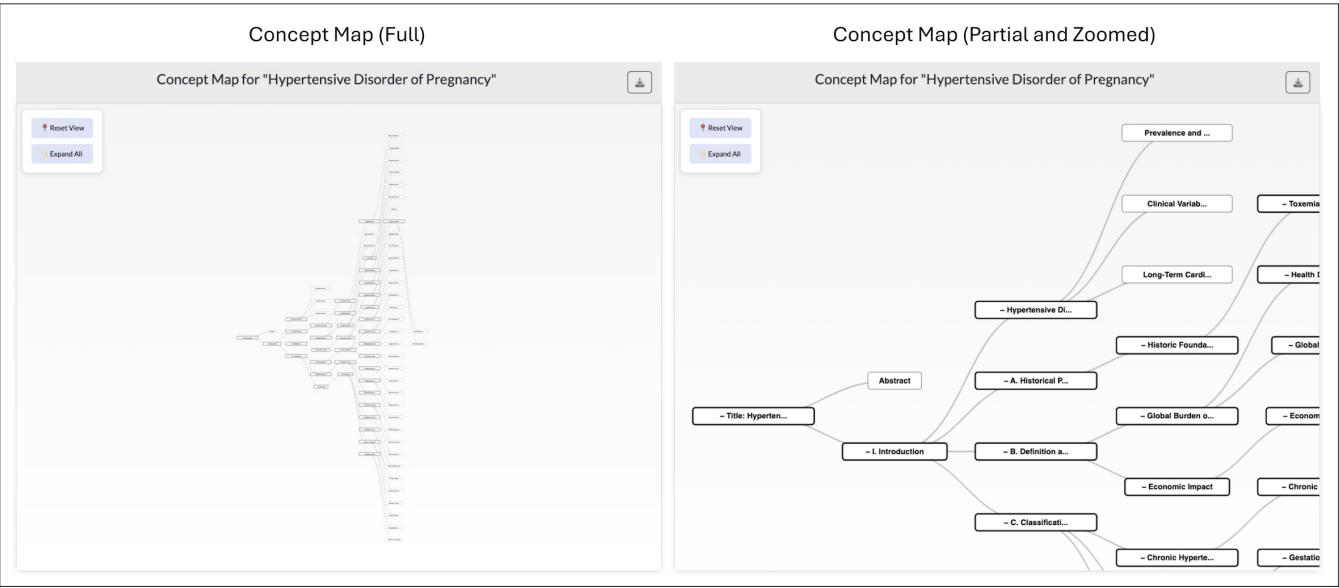


Figure 4. Concept map of a document generated by *Discourse2Draft*.

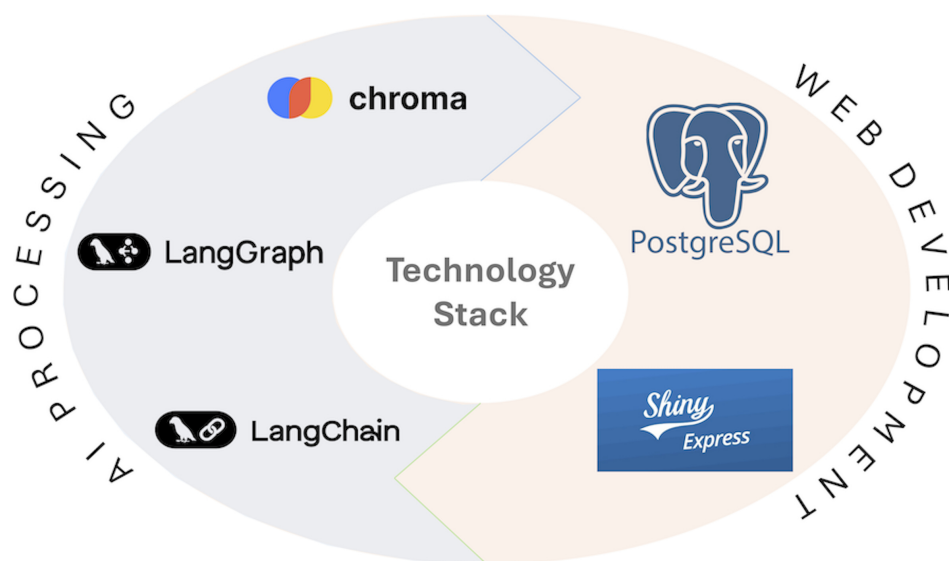


Figure 5. The technology stack used to develop the app.



Figure 6. Evaluation scores.